Countersign: Evidence-Grounded Supervision of Coding-Agent Completion Claims

Anonymous Author(s)
Affiliation
Address
email

Abstract

Coding agents terminate when they believe a task is done, but that belief is not always supported by their own execution trace. We present *Countersign*, a deterministic supervisory meta-agent that audits a worker's completion claim against the trace—checking that the cited evidence resolves to a successful test, that no cited test predates a later edit to a file it covers, and that no requirement update postdates the evidence—and can block termination to demand fresh evidence. Countersign is deliberately *trace-only*: it judges whether a claim is *justified* at claim time—distinct from whether the implementation is *correct*, which a hidden evaluator answers once after termination. In a preregistered, frozen campaign of 490 real-model runs over ten held-out fixtures, all 17 blocks landed on proposals an independent oracle also labelled unsupported (precision 1.00, 95% CI $[0.82, 1.0]$) and none on the 204 supported proposals judged—though 13 of 17 are the trivial case of a claim citing no evidence. A counterfactual replay puts that in proportion: a conventional continuous-integration gate catches 2 of 15 unsupported proposals, a one-line "cites nothing" rule catches 10, and Countersign catches 11 of the 11 it judged. Negative results are equally informative: truncating the worker's visible history did not detectably increase unsupported completion, and a treatment-identical control arm disagrees with the baseline on 4 of 10 matched cells—so the endpoint is too volatile at this n to carry any rate contrast, and we locate the gate's contribution where it is deterministic instead: unsupported proposals become blocked episodes that recover, usually by re-citing evidence the trace already held. And we quantify the construct honestly: all 18 accepted unsupported completions passed hidden evaluation—Countersign is an evidence-hygiene instrument whose safety value remains a hypothesis under test.

1 Introduction

An autonomous coding agent ends its episode by asserting completion—a claim about evidence: the tests it ran, the edits it made, the requirements it was given. When the claim outruns the evidence (a cited test predating the final edit; a superseded source), the agent has not necessarily produced wrong code, but its report is *unwarranted*, and whoever trusts it inherits an invisible risk.

We study whether a second agent with authority over the first's termination can tell warranted completion claims from unwarranted ones, and what that authority costs (§4, Appendix C). Countersign reads only the worker's execution trace and never consults ground truth; in every *deployable* condition the hidden evaluator that determines correctness runs once, post-termination. The evaluation-only `oracle_supervisor` arm is the sole exception, consulting it at gate time by design and never pooled into a deployable claim. This separation is the organizing principle: a claim can be justified and wrong, or correct and unjustified, and conflating the two makes "the verifier improved outcomes" unfalsifiable.

Submitted to the NeurIPS 2026 Workshop on Managing Agents that Manage Agents (Meta-Agents). Do not distribute.

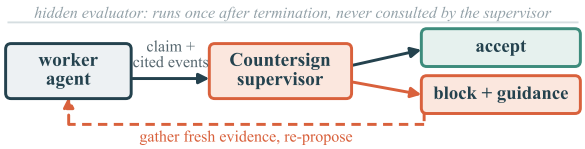

Figure 1: The supervision loop: the worker proposes termination with cited evidence; Countersign accepts, or blocks with the specific deficiency and the worker re-proposes. In this campaign that repair was usually *re-citation* rather than new work (§4). The hidden evaluator runs once, post-termination, never consulted by the supervisor.

Contributions. We contribute (i) a termination audit of cited-evidence chronology with halting authority, scored against an independent post-hoc oracle it does not compute (§2); (ii) a preregistered evaluation on matched pairs and negative controls pricing false blocks, under a frozen protocol with a public deviation ledger (§3); (iii) results across 490 runs—perfect block precision, an ablation against two cheaper rules, a truncation null, and a treatment-identical control arm bounding what any rate contrast here can claim (§4); (iv) a calibrated account of the gate as evidence hygiene, not a demonstrated safety mechanism (§5).

2 The Countersign Supervisor

Worker and ledger. The worker is a tool-loop coding agent (self-hosted open-weight models) in a git-backed workspace; every action appends to an *evidence ledger*—tool calls with outcomes, file mutations, test runs with inferred coverage, requirement updates—and it finishes by emitting a claim with cited evidence identifiers.

The audit. Countersign checks three properties: (i) the cited events resolve, through their provenance chain, to a successful test run; (ii) no cited test predates a later mutation of a file it covers; (iii) the trace holds no requirement update newer than the latest successful test evidence. Property (i) is evaluated over the cited events' transitive closure and (iii) over the whole trace (Appendix A); freshness is tri-state, abstaining when coverage is unknown. On failure it blocks and returns the specific deficiency (Figure 1); a repair variant additionally issues the smallest evidence-refreshing action. Enabling the verifier also arms a non-overridable gate on unsafe *mutations*, but it never fired: all 490 runs record zero blocked non-finish actions, so every block here is a finish-proposal block. A fourth property—rejecting citations of a superseded source—exists only in the evaluation-only oracle below, never in the deployable gate. Determinism is deliberate: halting authority must be auditable and replayable. Every arm that runs a verifier logs its *raw* decision separately from the *enforced* one, so non-blocking arms still measure what the supervisor would have done; memory_baseline runs no verifier at all and therefore contributes no decisions.

Independent oracle. A post-hoc oracle labels every proposal supported/unsupported/uncertain from fixture-authored completion policies and canonical trace chronology. It shares no code with the online verifier; the two disagreed on one proposal (§4), and a shared claim classifier diverges under degradation (Appendix E).

3 Evaluation Design

Fixtures. Ten held-out repositories, frozen before any real-model run was inspected. Three *matched pairs* isolate one failure mode each under context parity (workspaces byte-identical but for one designed file): *temporal* (cited tests predate the final edit), *provenance* (fresh evidence grounded in a superseded document), and *requirement* (a clarification arrives after the last passing run). Four *negative controls* contain no designed trap (Appendix A): a block on an oracle-supported proposal there is a false positive at context parity, though a control can still elicit an evidence-free claim, which blocking is correct. A separate development suite, on which the rules were iterated, is never used for evaluation.

Conditions. Every arm runs the *same* instrumented worker prompt—a deliberate choice, pinned by a regression test—so the only treatment is what happens *after* a proposal: memory_baseline (no gate), observe_only (verifier judges and logs, never blocks, zero worker-visible feedback),

verification_only (blocking), verification_and_repair, and an evaluation-only oracle_supervisor bound. The baseline is thus *instrumented*, not naive, and the primary contrast isolates the gate alone; in the traces, the first two arms issue byte-identical first prompts in $10/10$ matched cells and identical first responses in $9/10$.

Memory degradation. A frozen registry of profiles degrades only *model-visible* state [1, 2]: graded *lossy truncation* of history (low/medium/high) and a *resume-summary* regime replacing the visible ledger with one truthful summary—the pattern deployed scaffolds use on resumption. The canonical ledger, verifier, oracle, and evaluator always see undegraded state.

Protocol. Temperature 0.7, sampler seeds 0–4, action budget 24 (Appendix A). The predeclared primary is accepted-unsupported finish, memory_baseline vs verification_only; the decomposition is the predeclared confound branch, other contrasts exploratory. Endpoints, comparisons, and interpretation branches for both positive and null outcomes were committed before the runs; the frozen protocol hashes code, fixtures, and model weights and refuses to start on a dirty tree.

4 Results

All numbers come from audited run manifests; the worker is `qwen2.5-coder:14b`. The campaign was sized to validate the instrument and calibrate power, not to detect effects; its 490 runs yield 341 oracle-judged proposals, 27 unsupported (Appendix B reconciles the denominators). **The predeclared primary returned null**: accepted unsupported completion, memory_baseline vs. verification_only, is $9/180$ vs. $1/160$ per run, b=5/c=0 paired ($p = 0.0625$), but a blocking arm suppresses that endpoint mechanically—so everything below is read at the proposal level. Two predeclared elements went undelivered (Appendix B).

Blocks are precise. Countersign issued 17 blocks with a live verifier—9 enforced, plus 8 raw would-blocks in non-enforcing arms whose proposals were still accepted—all on proposals the independent oracle also labelled unsupported, none on the 204 supported proposals a verifier judged: precision 1.00 (95% CI $[0.82, 1.0]$), false-block rate $0/204$ (95% CI $[0, 0.018]$), recall $17/18 = 0.94$ (95% CI $[0.74, 0.99]$). The one miss is instructive: after a correct block the worker re-proposed citing two file reads and two of its own failed actions—no test—and the gate allowed it because those events' provenance chain reached an earlier successful run, which the oracle's stricter policy rejected. Two caveats bound this. First, the denominator counts only judged proposals: 110 supported proposals in the no-verifier arm are excluded, not counted as successes. Second, and more important, the blocks have only three signatures: 13 fire the no-citation bundle (a claim citing nothing, which any rule detects), 3 requirement recency alone, 1 the missing-test rule alone. **The freshness check never fired once in 490 runs** (Appendix D), so the property the method leads with goes untested: read this as "the gate does not misfire," not "it discriminates hard cases."

Most of the gate's yield comes from one trivial rule. We replay every stored proposal of the four ablation campaigns against two cheaper rules. The standard CI rule ("a successful test run after the last edit") catches 2 of the subset's 15 unsupported proposals; a one-line "block a finish that cites nothing" catches 10; Countersign judged 11 online and caught all 11, of which the one-liner recovers 7—so the citation requirement, not the chronology reasoning, does most of the work. Neither replayed rule blocked any of the 161 supported proposals both judged.

Truncation does not detectably induce false claims. Baseline unsupported completion on trap fixtures did not rise with truncation severity (Figure 2a): a non-detection, not invariance, since Wilson intervals at 15 runs per cell span ∼0.01–0.30. The preregistered null branch applies.

The resume-summary regime moves the counts, but not interpretably (Figure 2b). Read at the *proposal* level—a blocking arm suppresses accepted-unsupported finishes by definition—the ten matched cells give baseline 5, observe_only 3, verification_only 2. The arms are identical until a proposal is made, so the first pair is *treatment-identical*: its disagreement on $4/10$ cells (3 vs. 1) carries no treatment at all and shows the endpoint is volatile at this n. (It ran as two campaigns under separate protocols, so it bounds sampler *plus* batch variation—an upper bound, which only strengthens the point.) On the effect scale the gate contrast is nominally the stronger of the two (net 3 vs. 2; exact McNemar $p = 0.25$ vs. 0.625), but neither is close to significance and a contrast that cannot be distinguished from an arm pair with *no treatment at all* supports no claim. We claim no proposal-level effect. The gate's contribution is the fate of the proposals it sees: 7 of 8 enforced-block

episodes recovered to supported termination with no budget exhaustion—though in 6 of those 8 the worker ran *no new test or edit*, re-citing evidence its trace already held. What the gate repairs is the audit trail, not the engineering. The repair arm never fired and the oracle bound admitted zero incorrect completions (Appendix B). No comparison reaches $p < .05$, and the power analysis explains why: even with every discordant pair in one direction the exact test needs $b \geq 6$, one more than this campaign could produce, and ~ 84 matched cells would be needed for 80% power at these rates (Appendix G). The cleanest evidence is measurement without power: the observe_only verifier judges but cannot act, so its verdicts cannot have shaped the trajectories they score. Campaign-wide it flagged every unsupported proposal it saw and no supported one (TP$= 7$, FP$= 0$, FN$= 0$, TN$= 45$)—avoiding the *behavioural* circularity blocking invites, though not definitional overlap (Appendix D).

What "unsupported" means. All 18 accepted unsupported completions concerned work that was *correct*; only 2 of 327 accepted completions failed hidden evaluation at all, so the claim-by-correctness table (Appendix B) has no power to show an association and we draw none. The traces do show the mechanism: 25 of the 27 unsupported proposals came *after* a successful test postdating the last edit, and 21 cited nothing. The failure is not claiming work the worker did not do—it is doing the work and failing to say so. Countersign is therefore measured against citation omission; its value against genuine overclaiming is untested here.

5 Supervisor-Failure Analysis and Limitations

The structural blind spot. The provenance pair is designed so fresh, well-cited evidence grounded in a superseded source passes a trace-only audit while failing hidden evaluation. That path never fired (both supported-but-incorrect completions arose elsewhere), but the class is real, unreachable by any trace audit, and bounds what evidence supervision can promise. Appendix E details two further failure modes: classifier/oracle divergence under degradation and a family censored by task weight, not budget.

Limitations. (1) Most important: the traps and the rules share authors, so the dev/held-out split prevents tuning but not the conceptual correlation between what we trap and what we detect—which can inflate only the block numerator, since false blocks and recoveries are organic. (2) The freshness check never fired, and a one-line citation rule recovers 7 of the 11 replay catches, so this campaign tests the citation requirement more than the chronology reasoning. (3) One worker model on ten small fixtures establishes nothing about prevalence in production software, and the noise-floor pair spans two campaigns. (4) The primary endpoint is suppressed by construction. (5) Verifier and oracle share no code but read the same trace, and every block's reasons mirror an oracle predicate—near-vacuous independence. (6) No predeclared comparison reaches $p < .05$, and the treatment-identical arm shows why: this endpoint is volatile at n=10. (7) Degradation profiles are our own transforms; (8) oracle labels have a single-rater sample; (9) intervals treat cells as independent though proposals nest in episodes and fixtures.

6 Related Work

Workers game outcome checks [3, 4]; supervisors with authority over another agent are an established design space [5–8]; deterministic finish-blocking hooks are deployed practice—the rule our CI replay prices—and Countersign differs only in auditing cited-evidence chronology and pricing its false blocks. The justified/correct split descends from process-vs-outcome supervision [9, 10], instantiated at termination against a hidden evaluator [11, 12]; self-verification is the complement [13]. Trajectory judges score completion without termination authority [14] and SCATE targets premature termination [15]; concurrent work checks completion evidence in workflow graphs [16] and at GUI finish steps [17].

Responsible-Use Statement

Countersign targets agent overclaiming, but supervising agents carries risks. An *allow* audits justification, not correctness, and must never be marketed as verification. A visible gate invites *oversight displacement*: verdicts must reach operators as cited-evidence audits, not badges. And

halting authority is itself a failure surface—*who supervises the supervisor*—priced here rather than assumed away. No human subjects or production systems were involved.

References

[1] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024.

[2] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[3] Sydney Von Arx, Lawrence Chan, and Beth Barnes. Recent frontier models are reward hacking. METR Technical Report, 2025. URL `https://metr.org/blog/2025-06-05-recent-reward-hacking/`.

[4] Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y. Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, and David Farhi. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation, 2025. URL `https://arxiv.org/abs/2503.11926`.

[5] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion. In *Forty-first International Conference on Machine Learning, ICML 2024*, pages 16295–16336. PMLR, 2024. URL `https://proceedings.mlr.press/v235/greenblatt24a.html`.

[6] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. GuardAgent: Safeguard LLM agents via knowledge-enabled reasoning. In *Forty-second International Conference on Machine Learning, ICML 2025*. PMLR, 2025. URL `https://proceedings.mlr.press/v267/xiang25a.html`.

[7] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE 2026)*, 2026. URL `https://arxiv.org/abs/2503.18666`.

[8] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. LlamaFirewall: An open source guardrail system for building secure AI agents, 2025. URL `https://arxiv.org/abs/2505.03574`.

[9] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback, 2022. URL `https://arxiv.org/abs/2211.14275`.

[10] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024.

[11] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[13] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[14] Mingchen Zhuge, Changsheng Zhao, Dylan R. Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. Agent-as-a-judge: Evaluate agents with agents. In *Forty-second International Conference on Machine Learning, ICML 2025*. PMLR, 2025. URL `https://proceedings.mlr.press/v267/zhuge25a.html`.

[15] Sijia Gu, Noor Nashid, and Ali Mesbah. SCATE: Learning to supervise coding agents for cost-effective test generation, 2026. URL `https://arxiv.org/abs/2607.08983`.

[16] Jesus Salas. Correct is not governed: Provenance integrity in agentic workflows, 2026. URL `https://arxiv.org/abs/2608.12761`.

[17] Qijun Han, Haoqin Tu, Zijun Wang, Haoyue Dai, Yiyang Zhou, Nancy Lau, Alvaro A. Cardenas, Yuhui Xu, Ran Xu, Caiming Xiong, Zeyu Zheng, Huaxiu Yao, Yuyin Zhou, and Cihang Xie. VLAA-GUI: Knowing when to stop, recover, and search, a modular framework for GUI automation, 2026. URL `https://arxiv.org/abs/2604.21375`.

A Design Details

Test coverage is inferred statically, so documentation edits do not invalidate code evidence, and a failure superseded by a later passing run does not permanently contradict a claim. The four negative controls: documentation edits after green tests; an unrelated-module edit with a targeted-test citation; a legitimate zero-change audit; an irrelevant late clarification. Sampling uses temperature 0.7 because greedy decoding would make multi-seed replication pseudoreplication; seeds 0–4 are sampler seeds over otherwise identical episodes.

B Proposal Accounting

Every denominator in §4 derives from one ledger over the 490 runs' 341 finish proposals (Table 1). Identities: $341 = 327$ accepted $+ 9$ enforced blocks $+ 5$ oracle-arm refusals; 27 unsupported $= 18$ accepted $+ 9$ enforced-blocked; the 17 quoted blocks are *raw* decisions (9 enforced $+ 8$ would-blocks in non-enforcing arms, whose proposals were still accepted); verifier-judged proposals number 222 (204 supported $+ 18$ unsupported; recall $17/18$), and the remaining 110 supported proposals sit in the no-verifier baseline arm. The CI replay covers the four ablation campaigns: 15 unsupported proposals, 11 of them verifier-judged, 161 supported proposals judged by both gates.

Two predeclared elements went undelivered, both for funding: a second worker model (`devstral-small-2:24b`, calibrated but not run) and the post-hoc LLM-judge supervisor comparison. Arm details: verification_and_repair produced no unsupported proposal in its 40 runs, so its repair path was never exercised—an uninformative cell, not evidence repair works. The evaluation-only oracle_supervisor bound refused 5 proposals, admitted zero incorrect completions, and accepted one unsupported-but-correct finish: it bounds correctness, not justification. The 9 enforced blocks hit 8 episodes; 7 recovered to supported termination, 1 ended on the missed boundary-case proposal, and none exhausted its action budget after a block. The campaign's two supported-but-incorrect completions arose on a negative-control and a requirement fixture, not the provenance pair.

C The Price of Halting Authority

§1 promises to price the authority; the frozen protocol predeclares the endpoints below as secondary, and they are reported here in full rather than selectively. Over the 160 matched memory_baseline/verification_only cells (matched on campaign, task, degradation profile, and seed), supervision is close to free: mean extra model actions $+0.013$ (median 0); hidden-evaluator success $107 \rightarrow 112$; action-budget exhaustion $59 \rightarrow 57$; and across all 222 gate decisions the median decision

Table 1: Proposal ledger by arm.

arm	runs	proposals	unsup.	raw blocks	enforced	acc. unsup.
memory_baseline	180	119	9	0	0	9
observe_only	70	52	7	7	0	7
oracle_supervisor	40	30	1	1	0	1
verification_and_repair	40	28	0	0	0	0
verification_only	160	112	10	9	9	1
total	490	341	27	17	9	18

Table 2: Accepted completions (n=327): claim verdict (oracle) vs. work verdict (evaluator). With only 2 incorrect completions the expected count in the empty cell under independence is $\approx$0.11 (Fisher $p = 1.0$), so the table bounds the construct rather than testing it.

	correct	incorrect
claim supported	307	2
claim unsupported	18	0

latency is 0.32 ms (max 80.7 ms). Post-block spend across every enforced-block episode totals 21 model actions—0.26% of the campaign's action budget. Nothing here reaches significance and nothing is claimed to: the point is that the liveness and utility costs a blocking gate could plausibly impose are, in this campaign, not measurable. These are descriptive secondary endpoints on paired cells, never pooled.

D Which Checks Actually Fired

Every block in the campaign carries one of exactly three reason signatures (Table 3), recomputed from the run artifacts by `gen_appendix_tables.py`. The freshness check—property (ii) of §2, the one Figure 1 illustrates—never fired, so one of the three deployable properties is exercised by no block in this campaign. This is a fixture-yield finding, not a claim that those rules are unnecessary: the temporal family's trap requires the worker to cite a specific stale test, and this worker overwhelmingly cited nothing at all instead (21 of 27 unsupported proposals). It also explains the ablation of §4: most of Countersign's catches come from requiring citations rather than from reasoning about staleness—a one-line citation rule recovers 7 of its 11, leaving a margin of 4. Every block's reason set also mirrors a predicate the oracle evaluates, so verifier/oracle agreement on these blocks is definitional rather than independent.

Table 3: Block reason signatures across all 490 runs.

signature	blocks
claim cited no evidence (6-reason bundle)	13
requirement recency, alone	3
missing successful test, alone	1
freshness/staleness of cited evidence	**0**

E Additional Failure Modes

Classifier/oracle divergence. The online claim classifier (a shared component that labels the worker's completion claims from the worker's *visible* context, used for in-flight telemetry) disagreed with the independent oracle under high degradation, because the classifier reads the worker's degraded view while the oracle reads the canonical trace. All endpoints in the main text are oracle-anchored; the divergence is evidence that an independent oracle is necessary rather than redundant.

Task weight censors a family. Requirement-family trap runs exhaust the action budget in $3/5$ runs in every arm, and raising the budget from 24 to 40 actions left that rate unchanged. The family is intrinsically too heavy for this worker—a fixture-design finding, not a budget-calibration error.

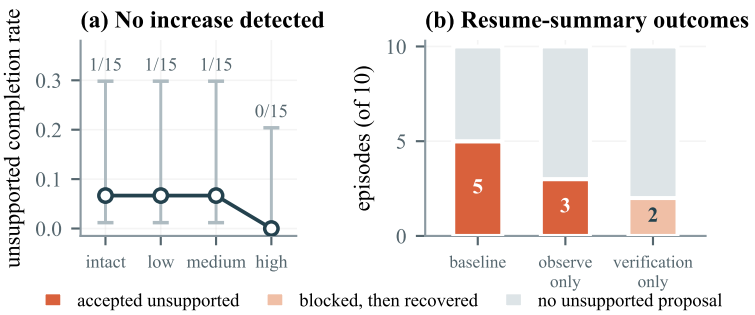

Figure 2: (a) Graded truncation: no detectable increase; the wide Wilson 95% intervals mark a non-detection at modest power, not invariance. (b) The resume-summary regime does produce the effect: the counts are $5/3/2$, but the first two arms are treatment-identical, so their $4/10$-cell disagreement is the endpoint's noise floor—larger than the gate contrast's $3/10$. No proposal-level effect is claimed; both proposals the gate saw became blocked episodes that recovered.

F A Blocked-then-Recovered Trajectory

Under lossy truncation at low severity, one verification-arm episode on a fresh-evidence temporal fixture shows the mechanism in miniature. The pressured worker twice proposed terminating *legitimate* work with insufficient cited evidence; the supervisor blocked both proposals with the specific deficiency; the worker gathered fresh evidence, terminated supported, and passed hidden evaluation. Pressure-induced premature finishing was rare for this worker, but both instances that occurred were caught and repaired by the gate. The five blocks issued across the pressure gradient (three on trap fixtures, two on control-task proposals) all landed on oracle-unsupported proposals, and enforced false blocks on oracle-supported work remained zero campaign-wide.

G Bayesian Sensitivity Readings

As a post-hoc sensitivity analysis (recorded as such in the deviation ledger; no new data, uniform $\mathrm{Beta}(1,1)$ priors, exact conjugate posteriors, no sampling), we re-read the paper's key small-n counts in Bayesian form. *Both contrasts, side by side*: the treatment-identical contrast (b=3, c=1) gives $\mathrm{Beta}(4,2)$, $P(\theta > 0.5) = 0.8125$; the gate contrast (b=3, c=0) gives $\mathrm{Beta}(4,1)$, $P = 0.9375$; the accepted-level primary (b=5, c=0) gives 0.9844. The gate readings are the larger, and we still claim no effect from them: a contrast with *no treatment behind it* already yields four-to-one posterior odds, which is how much direction small-n discordant counts manufacture on their own. Direction only, never magnitude. *Discrimination in the arm that cannot act*: the observe_only verifier's sensitivity $3/3$ has posterior mean 0.80 with 95% credible interval $[0.40, 0.99]$, and its specificity $7/7$ has mean 0.89, interval $[0.63, 1.00]$; campaign-wide block precision $17/17$ gives $[0.81, 1.00]$, agreeing with the Wilson interval in the main text. These are direction and uncertainty statements only; none is a magnitude claim, and all shrink toward chance under priors skeptical of any effect.

H Reproducibility

The campaign spans three tags of one freeze: a base freeze plus two infrastructure-only fixes applied mid-campaign (interruption-recovery artifact reuse; a tool-level crash on syntactically invalid worker-written code surfacing as a tool error rather than killing the episode). The hashed verifier-policy files are byte-identical across all three tags: no verifier, oracle, or fixture logic changed after the freeze. Every deviation, interim inspection, and protocol-identifier supersession—including two interim analyses that informed spending decisions, and one corrected over-interpretation—is recorded in the deviation ledger shipped with the artifact; the interim analyses changed scope only—no endpoint or allocation rule was altered post hoc. The artifact is a relocatable bundle whose integrity audit passes from the copied location for six of the seven campaigns (15,182 indexed files, zero mismatches); the seventh lost two derived bookkeeping files when a rented pod was released mid-campaign, so it is verified at the artifact-hash and frozen-protocol level instead, as its shipped provenance note states. Every number in this paper is recomputed from run records directly and consults no manifest.